

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

Relatório do Tutorial 8 / 9 Software Testing de Laboratório de Engenharia de Software (LES)

Diogo Emanuel Matos Honório **2021232043**

Introdution

This report presents the application of automated software testing techniques in Python, exploring unit test generation with Pynguin and mutation testing with Mutatest. The study evaluates the effectiveness of automatically generated test suites through coverage metrics and mutation scores.

Metrics for the test suite:

Based on the pynguin generated tests the metrics were the following:

- Line coverage: 71,97%
- Branch coverage: 37.10%
- Mutation score: 37.73%

```
Name                                 Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------------------
src\digitalWalletSystem.py             146     67     62      7    49%   316-320, 325-341, 346-349, 354-363, 368-380, 385-401, 406-415
tests\test_src_digitalWalletSystem.py   93      0      0      0   100%
--------------------------------------------------------------------------------
TOTAL                                  239     67     62      7    65%
```

Fig.1 – Results of Line and Branch Coverage


```
"summary": {
    "covered_lines": 79,
    "num_statements": 146,
    "percent_covered": 49.03846153846154,
    "missing_lines": 67,
    "excluded_lines": 0,
    "num_branches": 62,
    "num_partial_branches": 7,
    "covered_branches": 23,
    "missing_branches": 39
},
```


Fig.2 – Details of digitalWalletSystem.py results


```
"summary": {
    "covered_lines": 93,
    "num_statements": 93,
    "percent_covered": 100.0,
    "missing_lines": 0,
    "excluded_lines": 0,
    "num_branches": 0,
    "num_partial_branches": 0,
    "covered_branches": 0,
    "missing_branches": 0
},
```


Fig.3 – Details of test_src_digitalWalletSystem.py results

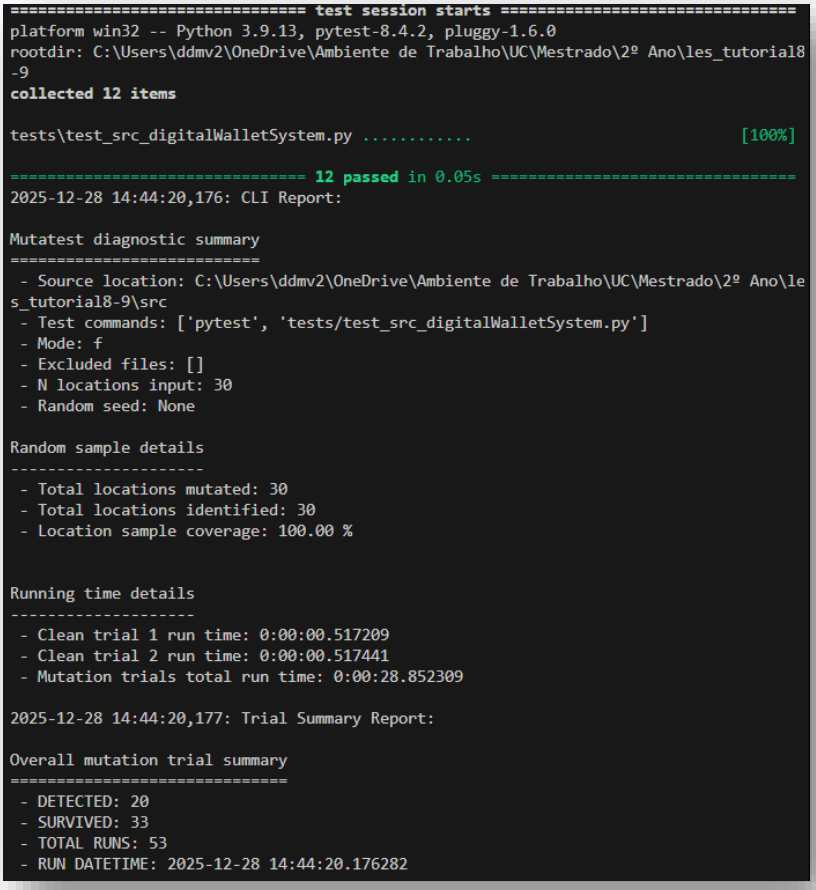


Fig.4 – Results of Mutation Score

Process description:

This report details the practical application of fundamental software testing methodologies as explored in Tutorial 8, adapted for the Python programming language. The following section provides an overview of the specific tools employed and a summary of their implementation within the testing workflow.

First, since everything is adapted for the Python programming language, the tool used is Pynguin. Pynguin is an automated unit test generation framework for Python, designed to create regression tests that achieve high code coverage for general-purpose Python programs. Therefore, the first step was to install Pynguin.

```
pip3 install pynguin
```

After that there is the need for the creation and activation of a Python virtual environment in order to isolate the dependencies and reproducible results.

```
python -m venv venv

venv\Scripts\activate
```

After all the setup is complete, we proceed to the pynguin test generation, with the following commands:

```
$env:PYNGUIN_DANGER_AWARE=1

pynguin --project-path . --module-name src.digitalWalletSystem --output-path tests
```

After the tests were generated by Pynguin, they needed to be executed. For this purpose, the pytest framework was used. Pytest is a Python testing framework that supports various types of software testing, including unit tests, integration tests, end-to-end tests, and functional tests. Therefore, the next step was to install pytest and use it to evaluate how many tests passed.

```
pip install pytest

pytest tests/test_src_digitalWalletSystem.py
```

Taking this entire process into consideration, we can conclude that the approach used to generate the test suite was fully automated, as all tests were generated by the Pynguin tool. The automation eliminated the need for manual test case design, reducing human effort and potential bias in test creation.

Finally, it was essential to collect the metrics, as this step is very important for evaluating how effective the generated tests were. To collect all the necessary metrics, the following commands were used.

```
coverage run --branch -m pytest tests/test_src_digitalWalletSystem.py

coverage report -m

coverage json
```

About mutation testing, there was a need to use a different tool besides mutmut since I am working on a Windows system. Taking this into consideration, I opted for a different tool called mutatest. This tool is a Python command-line tool for mutation testing that has the capability to assess the quality of unit tests by introducing bugs (mutants) into the code and checking if the tests catch them. As part of the process to perform mutation testing, it was necessary to create an environment with a Python version compatible with this tool (Python 3.9).

```
python -3.9 -m venv venv39

venv39\Scripts\activate
```

After this setup, we proceed to the mutation testing using the following commands:

```
pip install mutatest

mutatest -s src -t "pytest tests/test_src_digitalWalletSystem.py" -m f -n 30
```

This command will analyze all identified locations (30 in total, as specified by -n 30). Using the full mode (-m f) means that, even if a location contains multiple errors, all of them will be analyzed and reported, instead of stopping at the first error per location. This mode may lower the mutation score however makes the results more accurate and trustworthy (Figure 5 shows na example of the results without the full mode).

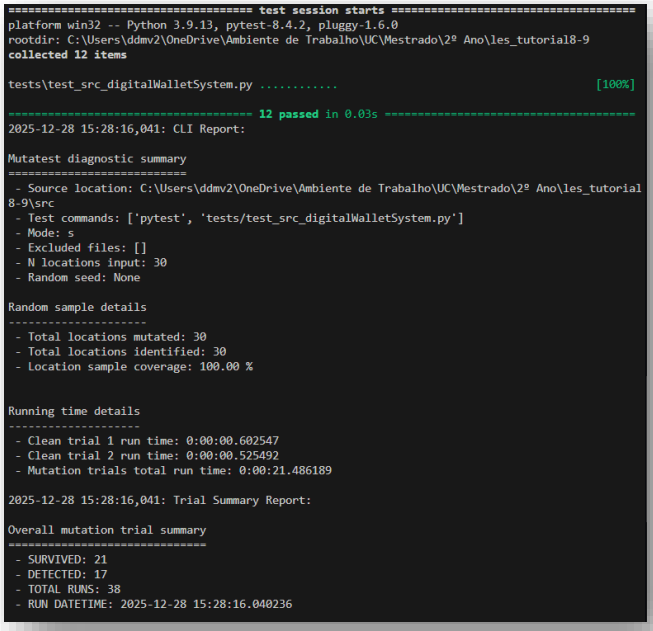


Fig.5 – Results of Mutation Score without full mode (44.73%)

Analysis of Results:

In this section, it will be made a detailed analysis in order to answer the requested questions.

- How effective was the submitted test at detecting bugs?

Based on the results presented previously, I would say that the performance was reasonably decent in terms of line coverage, showing a coverage of 71.97%. However, the branch coverage was far less effective, as only 37.10% of the branches were fully covered. Since this figure is below 50%, I would consider it relatively poor. This same effect applies to the mutation score, which was also below 50% (37.73%). I would argue that this metric is the most important for validating whether the tests are effective at detecting bugs, as that is its purpose.

To investigate more deeply how effective the generated tests are at detecting bugs, I decided to ask an LLM to generate a test suite of unit tests for my code and then analyze the coverage and mutation score of these new tests comparing to the ones generated by pynguin. The results are stated below:

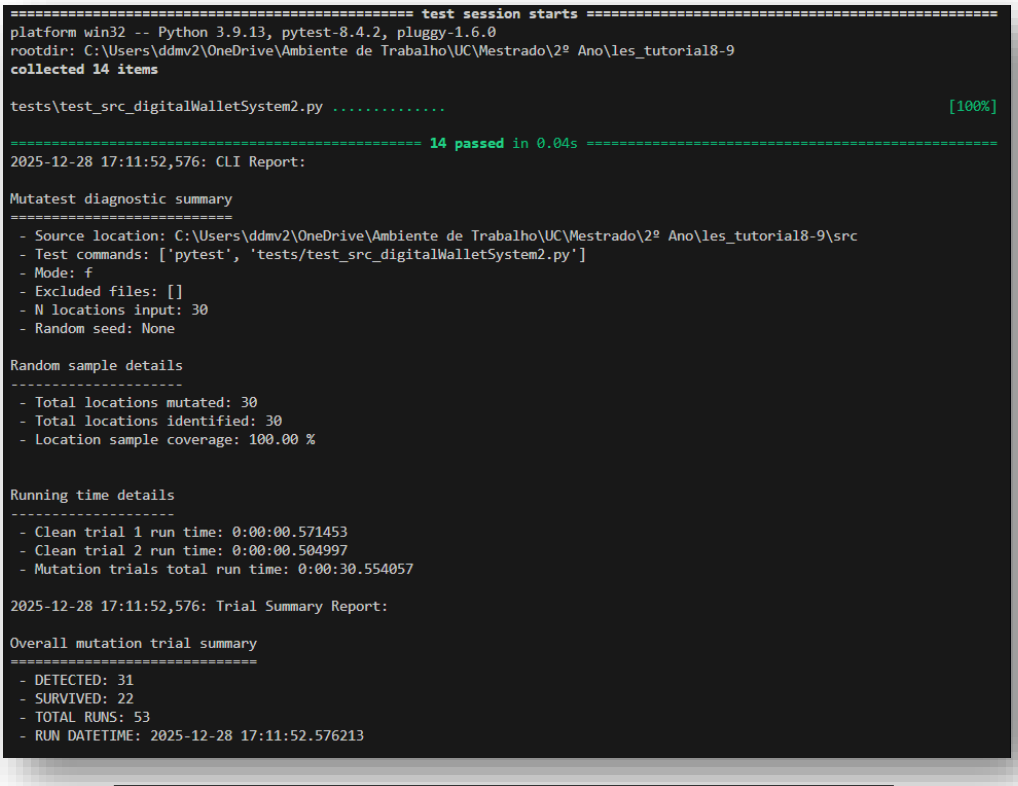
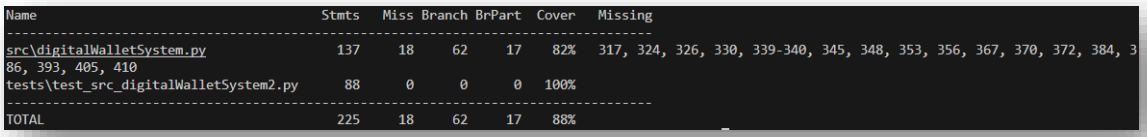
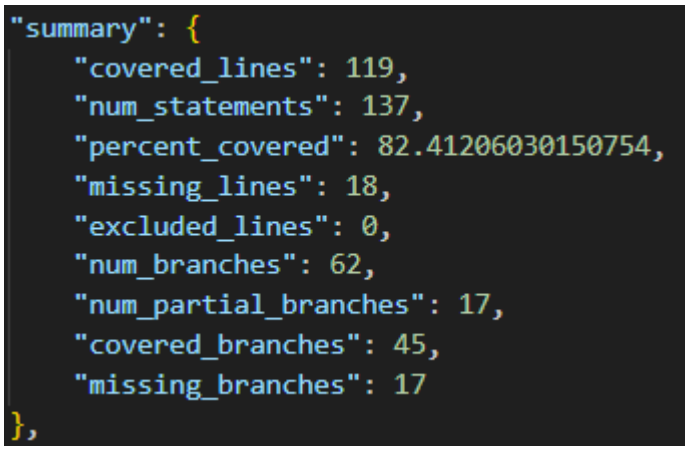


Fig.6 – Results of Mutation Score with full mode for LLM generated code (58.49%)



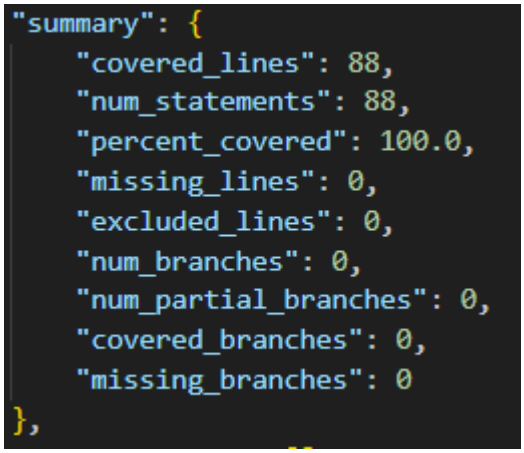
```
Name                                    Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------
src\digitalWalletSystem.py                137     18     62     17    82%   317, 324, 326, 330, 339-340, 345, 348, 353, 356, 367, 370, 372, 384, 3
86, 393, 405, 410
tests\test_src_digitalWalletSystem2.py     88      0      0      0   100%
-----------------------------------------------------------------------------------
TOTAL                                     225     18     62     17    88%
```

Fig.7 – Results of Line and Branch Coverage for LLM generated code



```
"summary": {
    "covered_lines": 119,
    "num_statements": 137,
    "percent_covered": 82.41206030150754,
    "missing_lines": 18,
    "excluded_lines": 0,
    "num_branches": 62,
    "num_partial_branches": 17,
    "covered_branches": 45,
    "missing_branches": 17
},
```

Fig.8 – Details of digitalWalletSystem2.py results



```
"summary": {
    "covered_lines": 88,
    "num_statements": 88,
    "percent_covered": 100.0,
    "missing_lines": 0,
    "excluded_lines": 0,
    "num_branches": 0,
    "num_partial_branches": 0,
    "covered_branches": 0,
    "missing_branches": 0
},
```

Fig.9 – Details of test_src_digitalWalletSystem.py results

Analysing these results for the LLM-generated code, we can say the metrics were as follows:

- Line coverage: 92%
- Branch coverage: 72.58%
- Mutation score: 58.49%

Taking these values into consideration, we can say that the LLM-generated code performed far better than the Pynguin-generated code, not only in line coverage where there was an improvement of 20.03%, but mainly in branch coverage where the percentage increased by 35.48%. Regarding the main metric for identifying bugs in the code, it is also possible to state an improvement in the mutation score, changing from 37.73% to 58.49%. After analysing all these metrics, I would say that the Pynguin-generated tests are not very effective at detecting bugs since they show a low mutation score that was easily surpassed by an LLM without human intervention.

- What are the limitations identified in the test suite for regression testing within a CI/CD pipeline?

Some noteworthy limitations include the low line and branch coverage. This is an important consideration, as this low coverage may compromise the CI/CD pipeline, some bugs might not be detected, thereby undermining the reliability of continuous

delivery. In addition to these limitations, it is important to note that the Pynguin tool is only capable of generating unit tests. However, a CI/CD pipeline must also include other types of tests, such as integration, system, and acceptance tests, to validate the overall system.

It is also worth mentioning that Pynguin uses random algorithms to generate tests. This means the tests do not always produce the same cases or values, resulting in variable coverage and thus compromising the regression testing process. Finally, the high resource cost and long execution times may violate the principles of continuous integration.

- What are the ways to adress the limitations stated above?

First, the main approach to address the limitation of low coverage would be to improve the tests generated by Pynguin. As a developer, I would say it is always necessary to analyze the generated tests and do more in order to get greater coverage, basically using the generated tests as a base for improvement.

Following this first approach, I would say the developer must also take into consideration creating the remaining needed tests, such as integration, system, and acceptance tests. All these tests are essential and must be created manually.

To mitigate the random tests generated by Pynguin, which result in various coverages, it is possible to define a fixed seed. This way, the tests will always be the same and have the same coverage. The limitation of time and resources is difficult to address, since it is possible to limit the time and number of test cases, but this might also make the tests less effective.

Why is mutation testing (and the mutation score) not widely used in industry, while line coverage receives more focus?

Mutation testing is not as widely adopted as line coverage. This is because mutation testing must run the test suite multiple times, once for each mutant introduced into the code, which can be a drawback in large projects, making the process less efficient and consuming significant time and resources.

As a result, line coverage is more commonly adopted than mutation testing due to its speed. Although line coverage is not perfect, it serves as a basic and practical way to test code. While imperfect, the return on investment for line coverage is often considered more worthwhile compared to mutation testing.